Q: questions or concerns?

Lecture 12: Indirect Communication (part 1)

COMP2014 Distributed Systems

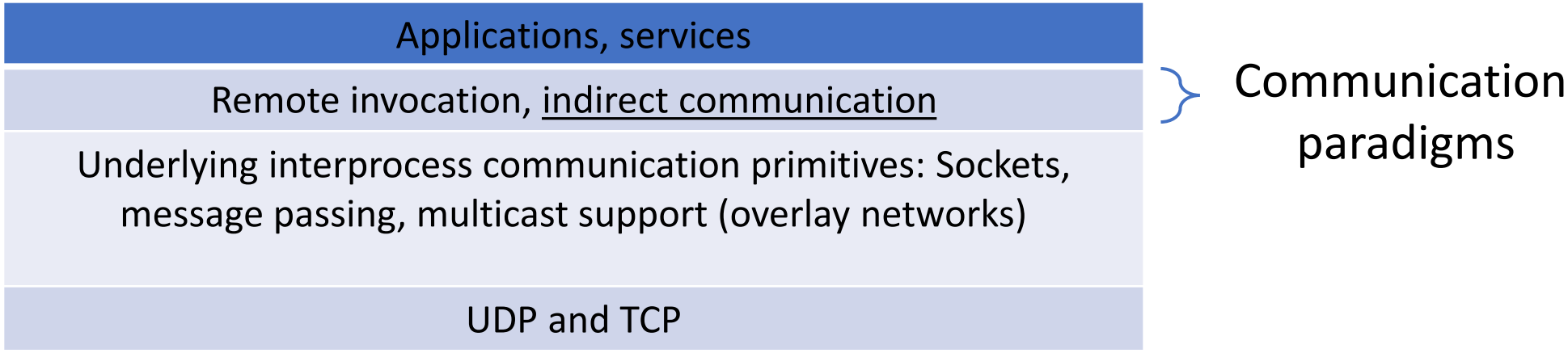
Chris Greenhalgh

School of Computer Science

Contents

- Indirect communication
- Publish-subscribe
  - Inc. filtering models & centralized vs distributed implementation
- Group Communication
  - Inc. message ordering & membership management
- Next lecture:
  - Message Queues
  - DSM
  - Tuple spaces
  - Redis

Indirect communication



Indirect communication - definition

- Def: “communication between entities in a distributed system through an intermediary with no direct coupling between the sender and the receiver(s).”
- Contrast the **direct** communication
  - E.g. Request-Reply, RPC, RMI
  - Where communication is:
    - one-to-one,
    - to a known/specified receiver,
    - running at the same time

Indirection (in computer science)

Q: Other examples of indirection in computer science?

- “All problems in computer science can be solved by another level of indirection.”
  - Attributed to Roger Needham, Maurice Wilkes and David Wheeler
- “There is no performance problem that cannot be solved by eliminating a level of indirection.”
  - Attributed to Jim Gray
- ☺

Space and time uncoupling

May be uncoupled in space and/or time:

- **Space uncoupling**:
  - the sender does not (need to) know the **identity** of the receiver(s) and vice versa
  - E.g. participants can be replaced, updated, replicated, migrated
- **Time uncoupling**:
  - the sender(s) and receiver(s) can have independent **lifetimes**
    - i.e. they do not need to exist at the same time
  - E.g. in a volatile environment

Space and time coupling in distributed systems

Q: Anything missing?
Or that doesn't fit?

	Time-coupled	Time-uncoupled
Space-coupling	*Properties:* Communication directed towards a given receiver or receivers; receiver(s) exist at that moment in time *Examples:* message passing, remote invocation	*Properties:* communication directed towards a given receiver or receivers; sender(s) and receiver(s) can have independent lifetimes. *Examples:* Queued RPC, DTN (next slide)
Space-uncoupling	*Properties:* Sender does not need to know the identity of the receiver(s); receiver(s) exist at that moment in time *Examples:* IP multicast, group communication, Pub-sub, DSM	*Properties:* Sender does not need to know the identity of the receiver(s); sender(s) and receiver(s) can have independent lifetimes. *Examples:* Message queues, tuple spaces, persistent event streams

Aside: Time uncoupling vs asynchronous communication

- **Asynchronous** message passing = “synchronisation uncoupling”
  - NOT time uncoupling:
  - The receiver must exist when the message is sent
  - (although the sender doesn’t wait)
- With time uncoupling the message itself **persists**
  - E.g. awaiting a currently unavailable receiver
  - E.g. Delay Tolerant Networking, Internet email, queued-RPC

Indirect communication paradigms

- A range of paradigms where
  - a message is *not* addressed to one specific entity, but
  - is sent to many entities and/or
  - is sent via some intermediary
- For example, this lecture:
  - Publish-subscribe (pub-sub) systems, Group communication
- Next lecture
  - Message queues, Tuple spaces, Distributed Shared Memory (DSM)

Communicating entities and communication paradigms

(the "menu")

Communicating entities (what is communicating)		*Communication paradigms (how they communicate)*		
System-oriented entities	*Problem-oriented entities*	*Interprocess communication*	*Remote invocation*	*Indirect communication*
Nodes	Objects	Message passing	Request-reply	Group communication
Processes	Components	Sockets	RPC	Publish-subscribe
	Web services	Multicast	RMI	Message queues
				Tuple spaces
				DSM

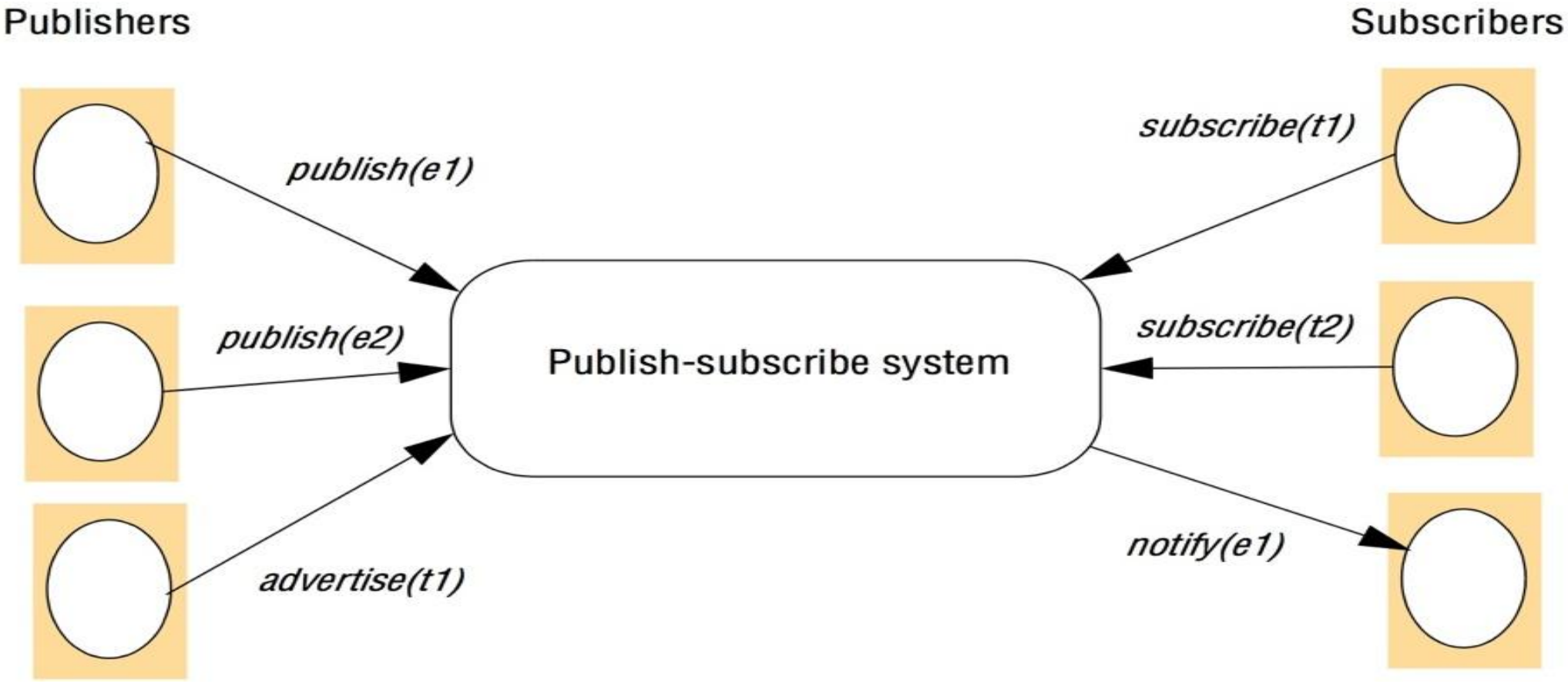
Publish-subscribe

Publish-subscribe model

AKA distributed event-based systems

- **Publishers** publish structured **events**
  - Operation *publish(e)*
- **Subscribers** express interest in particular events
  - in terms of **subscriptions** or **patterns**
  - Operation *subscribe(e')*
- An **event service** matches published events to subscriptions,
  - **notifying** subscribers accordingly
- Optionally,
  - publishers may also **advertise** that they produce certain types of event

The publish-subscribe paradigm



Applications of pub-sub

Q: has anyone used one? Why?

- Live feeds of realtime data
  - E.g. likes, chat messages
- Monitoring applications,
  - e.g. network monitoring
- Automated trading systems
- Ubiquitous computing,
  - e.g. responding to location events
- Cooperative work, e.g. shared document editing

- For example:
  - https://www.pubnub.com/
  - http://socket.io/
  - https://redis.io/ & https://redis.io/docs/latest/commands/redis-8-6-commands/#pubsub-commands => see next lecture & following lab
  - https://azure.microsoft.com/en-gb/products/event-grid/
  - https://azure.microsoft.com/en-gb/products/web-pubsub/

Pub-sub characteristics and options

- (Often) Loosely coupled systems…
- **Heterogeneity**
  - Publishers and subscribers can be created with different technologies and completely independently of each other
- **Asynchronicity**
  - Publishing an event is almost always asynchronous,
    - i.e. publishers do not wait for subscribers
- **Delivery guarantees**
  - Systems can vary in **reliability** (e.g. reliable or best effort) and **ordering** of events
    - Typically **FIFO ordering,** i.e. messages from one publisher are delivered to a subscriber in the same order in which they were sent
    - (See group communication message ordering options)

Subscription (filter) model

= how subscriptions are expressed

- i.e. how events are matched/filtered to subscribers

Roughly in order of complexity/flexibility:

- **Channel-based**: each event is published to a specific channel
  - subscribers subscribe to channels
- **Topic-based**: each event has a topic (or subject)
  - subscribers identify topic(s) of interest
- **Content-based**: each event has structured content (e.g. named fields)
  - subscribers specify patterns to be matched by each event

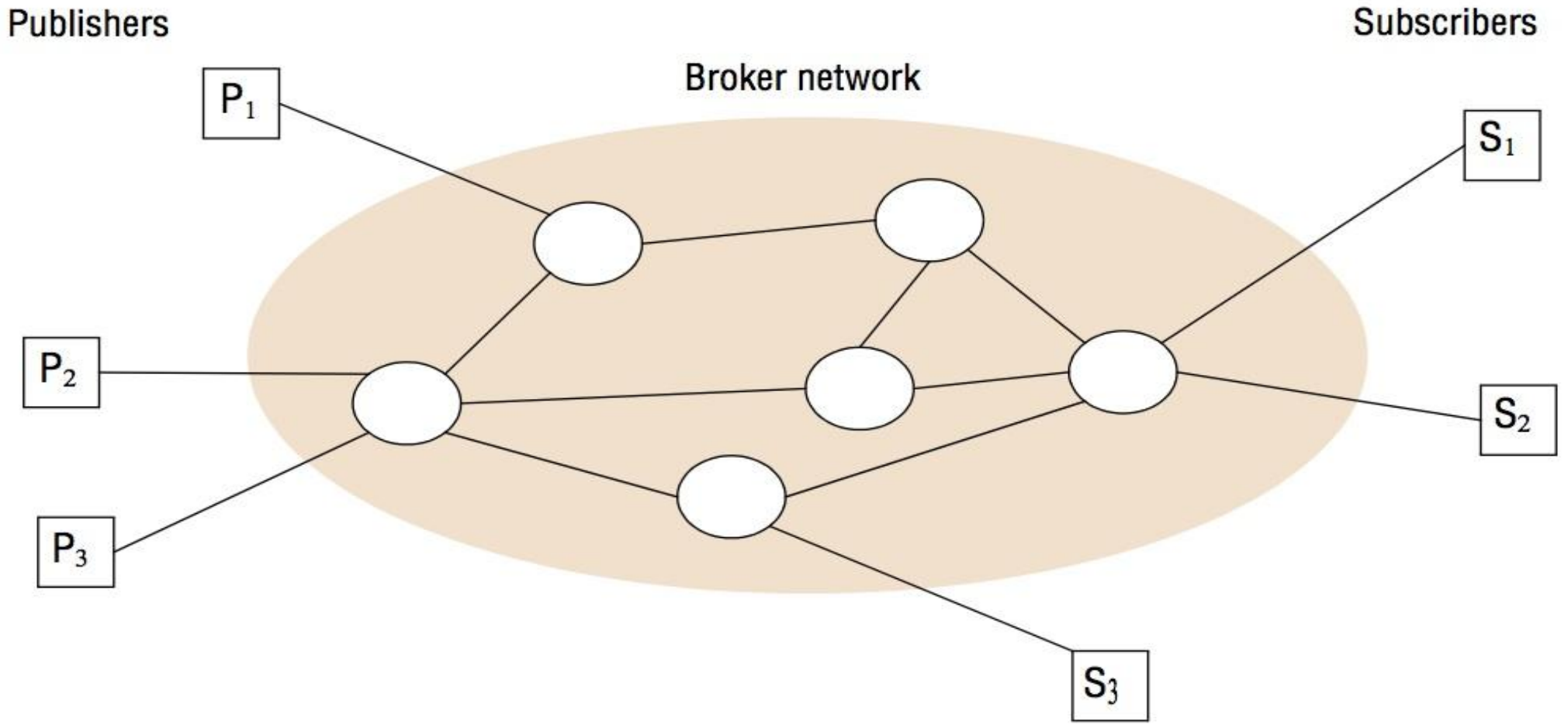
Other less common subscription models

- **Type-based**: events are typed
  - Subscribers specify types of interest
    - May be object-oriented, i.e. using common super-classes or interfaces
- **Objects of interest**: events are linked to a specific object
  - Subscribers specify the object(s) of interest
    - E.g. monitoring changes in a shared object
- **Semantic filtering**: events are described using a formal ontology
  - Like content-based filtering with additional inference, i.e. equivalence rules
    - Intended to describe the “meaning” of events
- **Complex event processing**: subscriptions are based on more than one event
  - E.g. event A happens shortly after event B

Centralised vs distributed implementations

- A **centralised** event service:
  - Is relatively easy to construct
  - The event service becomes a bottleneck
    - And single point of failure
- A **distributed** event service
  - Is more complex
  - But may be more scalable,
    - more reliable
  - E.g. a network of federated **event brokers** passing events to each other…

A network of event brokers



Instructor's Guide for Coulouris, Dollimore and Kindberg Distributed Systems: Concepts and Design Edn. 4

Event routing in a distributed event system

Event brokers have to decide how to pass on or **route** events, e.g.

- **Flooding**: every event is sent to every event broker
- **Filtering**: event brokers share subscription information and forward events to where valid subscriptions exist
  - **Advertisements** from publishers may help to filter subscription information for forwarding
- **Rendezvous**: there is a way to identify particular event brokers to handle matching events and subscriptions
  - So the corresponding events & subscriptions are all sent to it

Group Communication

Group communication overview

Q: has anyone used one? Why?

- An application-level abstraction of **multicast** communication
  - (See later lecture)
  - May be implemented over IP Multicast but doesn't have to be
- Typical applications:
  - Fault-tolerance, e.g. replicated services or data
  - Collaborative applications, e.g. shared whiteboard
- For example:
  - http://www.jgroups.org/

Group programming model

- There are **groups**…
- that **processes** can **join** or **leave**
  - (some group communication systems support groups of **objects** rather than groups of processes)
- A **message** sent to a group is delivered to all **members** of that group
  - i.e. operation *aGroup.send(aMessage)*

Group communication options

- **Closed** and **open** groups
  - Non-members cannot send to closed groups
- **Overlapping** and **non-overlapping** groups
  - non-overlapping => each process can be a member of at most one group at any moment
- **Synchronous** and **asynchronous** communication
  - Synchronous *group* communication => does a sender blocks until *all* group members have received the message/replied?

Implementation issues: Reliability

- Group Communication services are typically reliable
  - Or have the option to choose reliable and unreliable delivery per-message (unlike network-level multicast which is best effort)
- Reliability for group communication extends that for direct communication with **Agreement**
  - i.e., if the message is delivered to one process, then it is delivered to all processes in the group.
  - (See also later lectures on Reliability and Failure)
- But what if the processes in the group change over time?
  - Which ones need to receive it for agreement??
    (see Group Membership Management slide)

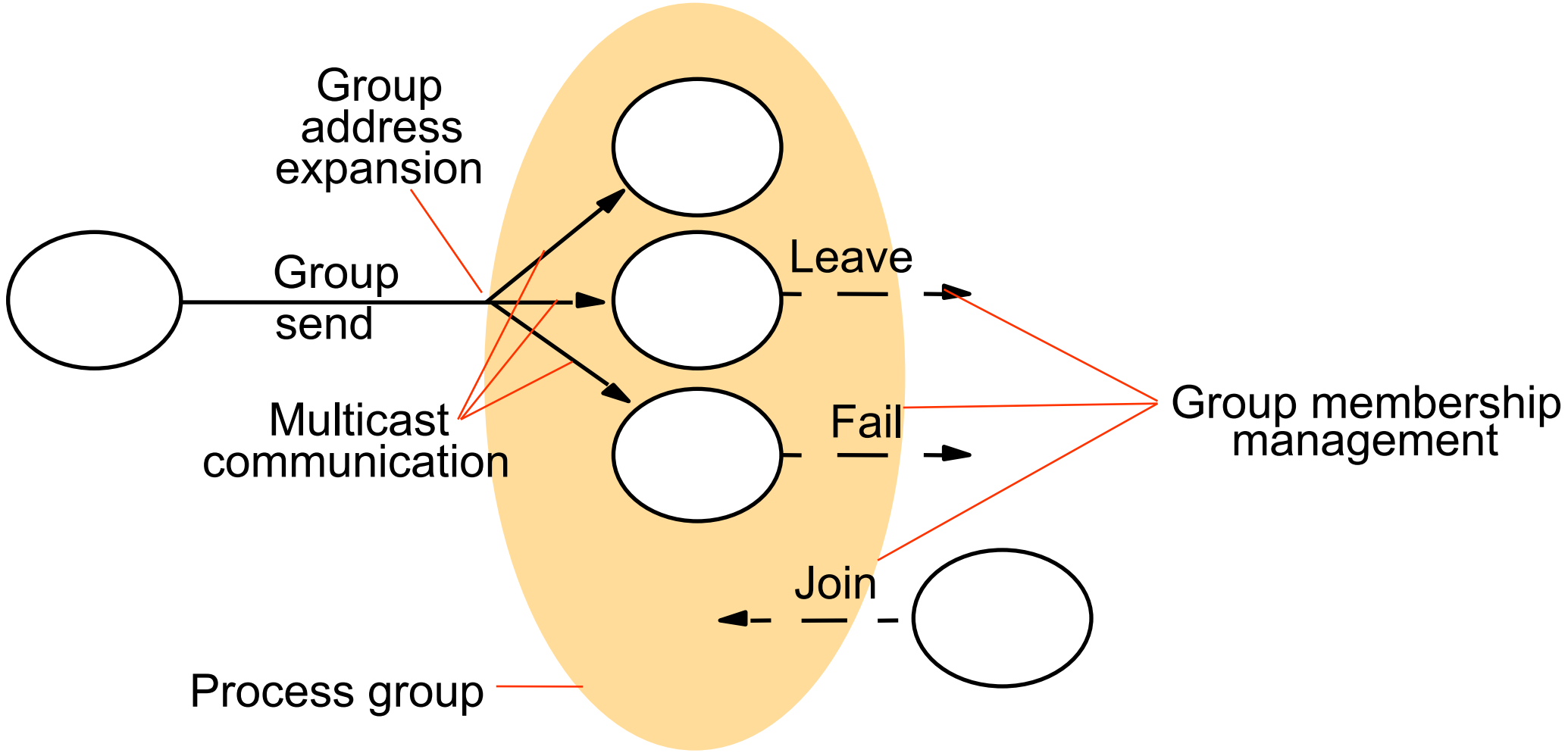
Implementation issues: Ordering

- What (if any) ordering constraints are imposed on delivered messages
  - Typically a trade-off:
    - stricter ordering constraints => longer delays and (for total order) more messages
- **FIFO**
  - Messages from one sender are delivered in the same order in which they were sent
    - But messages from different senders may be interleaved in different ways
- **Causal order**
  - If one message “happens before” another then it is delivered first
  - Like logical time – see later lecture on Distributed Algorithms
- **Total order**
  - All messages are delivered in the same order in every process

Implementation issues: Group membership management

- Group membership is fundamental
  - the system must be able to map groups to members
- Normally requires an explicit management interface
  - i.e. to join and leave group(s)
- For reliable communication there must also be failure detection, e.g.
  - member crashes => must be removed from the group,
  - network partitions => group partitions or half of it fails
- The group may (or may not) provide notification of group changes
  - Which may be treated as group messages, e.g. ordered, reliable, …

The role of group membership management



Indirect Communication Summary

- **Indirect communication** is
  - “communication between entities in a distributed system through an intermediary with no direct coupling between the sender and the receiver(s).”
- May be **uncoupled** in
  - **Space**, i.e. specific identities of senders and receivers are not known
  - **Time**, i.e. senders and receivers have independent lifetimes.
- For example:
  - **group** communication, **publish-subscribe**, **message queues**, **tuple spaces**, **distributed shared memory** (DSM)

Publish-Subscribe Summary

- In **publish-subscribe** systems,
  - **publishers** publish **events** to an event service
  - which filters and delivers message to relevant **subscribers**
- Subscriptions can be based on:
  - **Channel**, **topic**, **content**, **type**, **semantic** match, **objects of interest** or **complex** events (i.e. multiple events)
- Implementation may be **centralised** or **distributed**
- Can support very open and flexible systems
- For example:
  - https://www.pubnub.com/
  - http://socket.io/
  - https://redis.io/ => see next lecture & following lab

Group Communication Summary

- **Group communication** =>
  - Sending messages to a **group** rather than specific recipients
  - Processes can **join** and **leave** the group,
  - All current group **members** receive each message
- Group communication can vary, e.g.
  - **open**/closed groups,
  - non/**overlapping** groups,
  - **reliable**/unreliable message delivery and
  - **FIFO**, **causal** or **total ordering** of messages
  - (Can be implemented over IP multicast for communication efficiency but need not be)
- For example:
  - http://www.jgroups.org/

Next steps

- Next lecture:
  - Indirect communication part 2, i.e.
    Message Queues, DSM, Tuple spaces & Redis
- Following lab:
  - Indirect communication with Redis